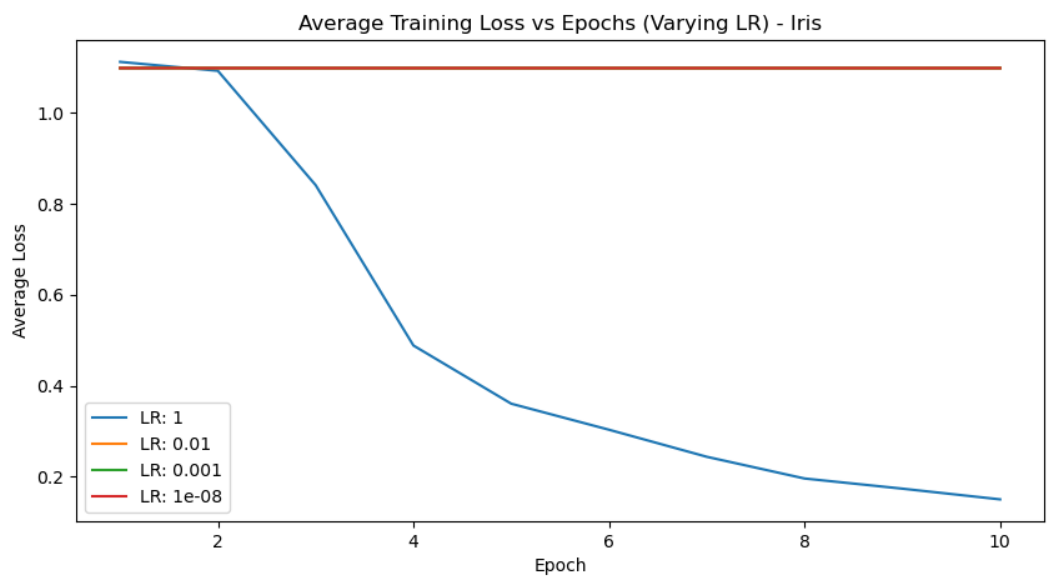
CS 639: Intro to Foundation Models
Homework 1

Amogh Dixit (asdixit3)

Part 1: Iris Dataset

(a)



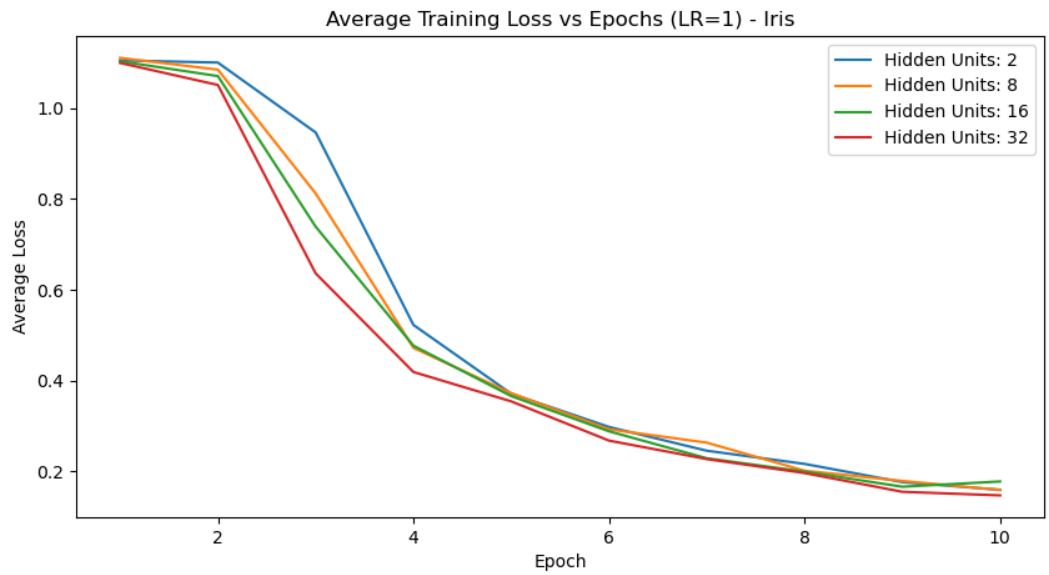
(b) According to the above plot, a learning rate of 1 produces the best model, since the average loss converges to a significantly lower value compared to the other learning rates. This aligns with the expected behavior because we are only training for 10 epochs, smaller learning rates like 0.01 and 0.001 fail to converge quickly enough (possibly because of how they are initialized, i.e. U(-0.01, 0.01)), leaving the model heavily underfitted.

(c)

Learning Rate	**Train Loss**	**Test Loss**
1	0.1501	0.1523
0.01	1.0986	1.0091
0.001	1.0986	1.0986
1e-8	1.0986	1.0986

According to the loss metrics, a learning rate of 1 produces the best model, achieving the lowest train loss of 0.1501 and test loss of 0.1523. This aligns with the expected behavior because we are only training for 10 epochs, smaller learning rates like 0.01 and 0.001 fail to converge quickly enough, leaving the model heavily underfitted.

(d) I used learning rate = 1 instead of 1e-2 as mentioned in the instructions since lr=1 was generating a much smoother curve, and we were asked to use the best LR to generate the curve on Piazza.



(e) Here are the average training losses, test losses, and test accuracies for each of the four models:

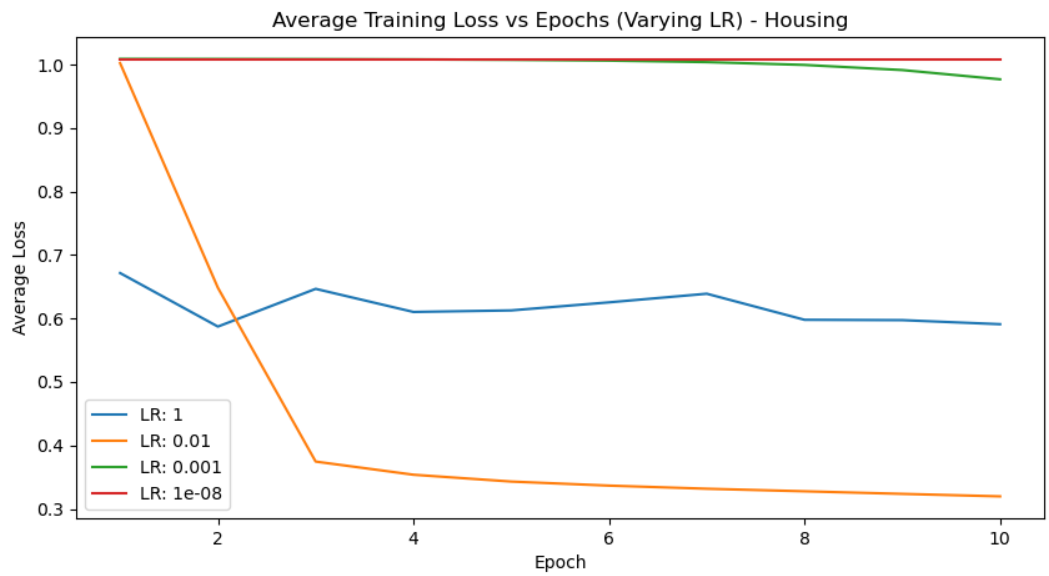
Hidden Units	Training Loss	Test Loss	Test Accuracy
2	0.1600	0.1588	0.9333
8	0.1595	0.1582	0.9667
16	0.1783	0.1615	0.9667
32	0.1478	0.1401	0.9667

Patterns Observed:

- The test accuracy increases marginally as we move from 2 to 8 hidden units. Because the Iris dataset is relatively simple and linearly separable, even a small network with 2 hidden units possesses enough capacity to accurately classify the species.
- The training loss fluctuates slightly (0.1600 $\rightarrow$ 0.1595 $\rightarrow$ 0.1783) before reaching its global minimum of 0.1478 at 32 hidden units. The test loss follows a similar pattern, achieving its absolute lowest point of 0.1401 at 32 hidden units.
- Although the classification accuracy is saturated, the network with a higher capacity of 32 hidden units allows it to output probability distributions that are closer to the true one-hot labels, thereby having the least loss.

Part 2: California Housing Dataset

(a)



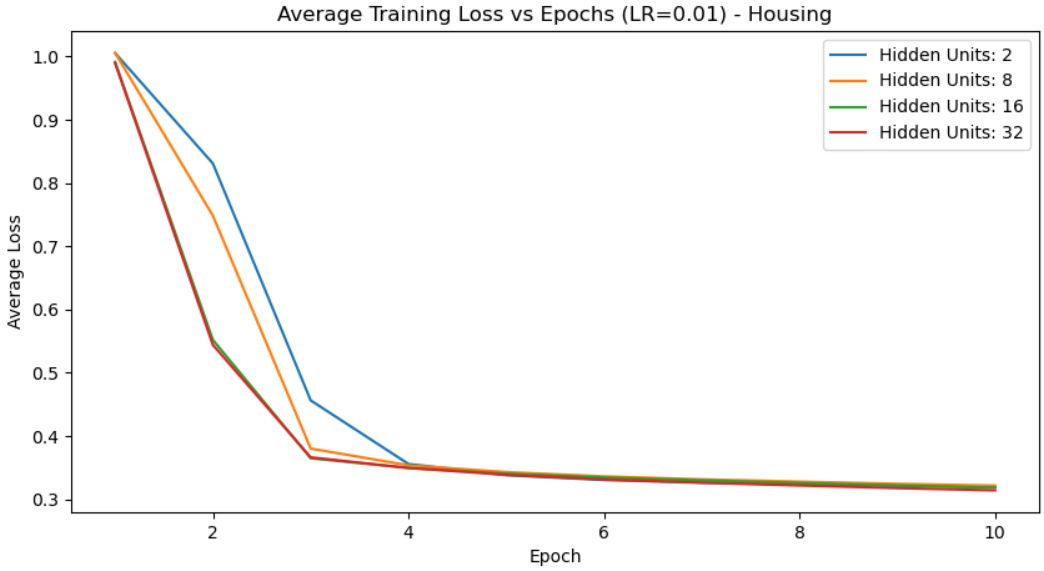
(b) For this regression task, a learning rate of 0.01 produces the best model. A learning rate of 1 is too aggressive for this dataset, likely causing the gradients to overshoot the minimum. Conversely, the much smaller learning rates (1e-3 and 1e-8) update the weights too slowly to converge within the 10 epoch limit.

(c)

Learning Rate	**Train Loss**	**Test Loss**
1	0.5910	0.6282
0.01	0.3197	0.3013
0.001	0.9769	0.9226
1e-8	1.0091	0.9635

According to the test set loss, the learning rate of 0.01 produces the best model, achieving the lowest MSE. This perfectly aligns with the findings produced by the training set losses, confirming that an LR of 0.01 offers a stable and effective convergence rate without overshooting or stagnating.

(d)



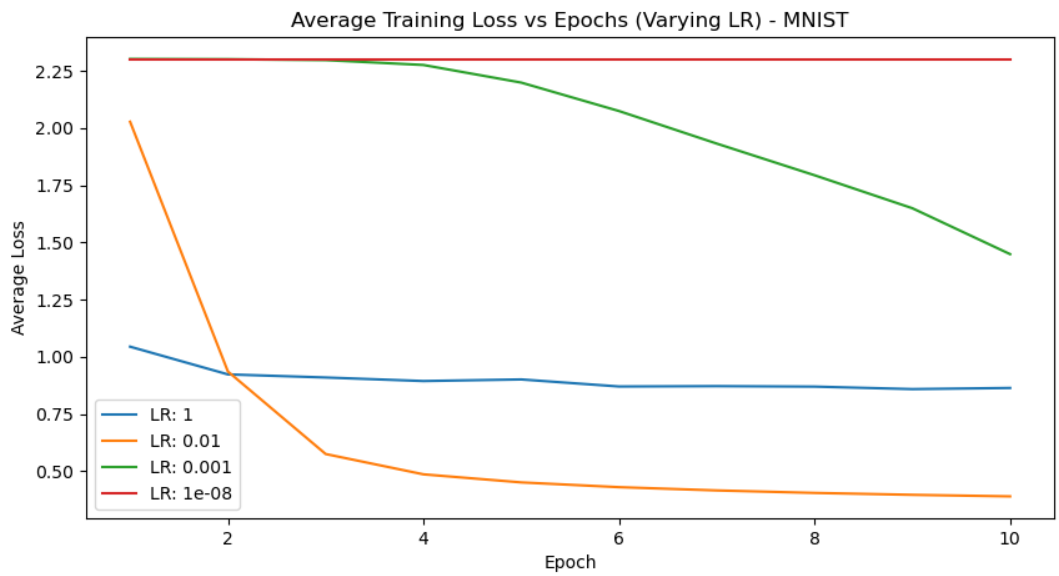
(e) Here are the average training losses and test losses (MSE) for each of the four models:

Hidden Units	**Training Loss (MSE)**	**Test Loss (MSE)**
2	0.3194	0.3071
8	0.3216	0.3058
16	0.3182	0.2999
32	0.3141	0.2953

Patterns Observed: As the number of hidden units increases, both the training and test losses consistently decrease. The added model capacity allows the neural network to capture the more complex, non-linear relationships present in the California housing features. The network with 32 hidden units achieves the lowest error on both the training and test sets, showing no immediate signs of severe overfitting.

Part 3: MNIST Dataset

(a)



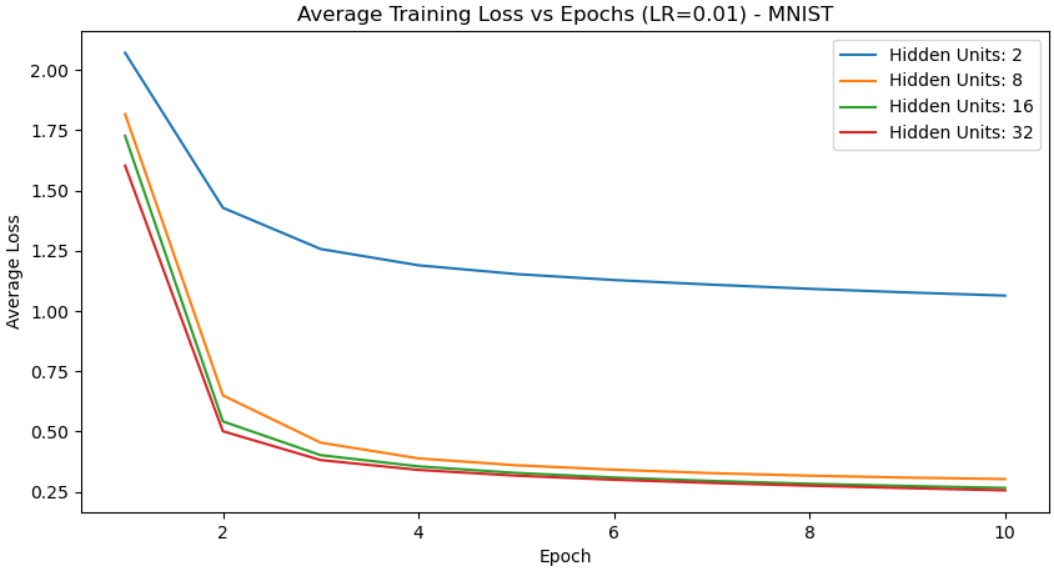
(b) According to the training loss plot, a learning rate of 0.01 produces the best model. It shows a steep and smooth descent to the lowest training loss value. A learning rate of 1 proves too aggressive, plateauing at a high loss, while the smaller learning rates (1e-3 and 1e-8) converge far too slowly to be effective in only 10 epochs.

(c)

Learning Rate	**Train Loss**	**Test Loss**
1	0.8644	0.7377
0.01	0.3907	0.3813
0.001	1.4493	1.3182
1e-8	2.3026	2.3026

According to the test set metrics, the learning rate of 0.01 produces the best model by a significant margin. This corresponds exactly with the training set findings, where LR 0.01 demonstrated the most optimal loss.

(d)



(e) Here are the average training losses, test losses, and test accuracies for each of the four models:

Hidden Units	**Training Loss**	**Test Loss**	**Test Accuracy**
2	1.0641	1.0590	0.6221
8	0.3028	0.3019	0.9158
16	0.2659	0.2594	0.9278
32	0.2558	0.2482	0.9287

Patterns Observed:

- There is a massive jump in performance (accuracy increases from 62.21% to 91.58%) when moving from 2 hidden units to 8 hidden units. Because the MNIST dataset requires mapping 784 input pixels to 10 distinct classes, restricting the hidden layer to 2 units creates a severe bottleneck.
- Performance continues to improve as model capacity increases up to 32 units, with both training and test losses steadily decreasing. However, the marginal gains in accuracy become much smaller after 16 units (92.78% $\rightarrow$ 92.87%), indicating diminishing returns for this simple single-hidden-layer architecture.

Part 4: Dataset Comparison

Across the three tasks, the Iris dataset is definitively the easiest for the network to learn, while the MNIST dataset appears to be the most difficult.
The ease of the Iris dataset is heavily supported by the experiments. The model achieves a near-perfect test accuracy of 96.67% with a test loss of 0.1332 using merely 2 hidden units. The learning curves for Iris also demonstrate extremely rapid convergence, often settling into a low-loss minimum within the first few epochs.
Conversely, the MNIST dataset struggles severely when capacity is constrained. With only 2 hidden units, the network achieves a meager 62.21% test accuracy and a high test loss of 1.0590. It requires a significantly larger capacity (at least 16 to 32 hidden units) to surpass 92% accuracy. Furthermore, both the MNIST and California Housing datasets require a much more carefully tuned learning rate (0.01) to ensure stable convergence, whereas Iris easily converges even with a highly aggressive learning rate of 1.

This disparity is primarily due to the dimensionality and intrinsic complexity of the data. Iris dataset is very simple. It contains only 4 numerical input features and 3 output classes, and the classes are largely linearly separable. On the other hand, Housing dataset features have non-linear relationships. Predicting exact median house values requires the model to learn nuanced interactions between the features making it harder than simple classification. MNIST is mathematically much more complex. Each input is a 28×28 image flattened into a 784-dimensional vector, mapped to 10 highly varied digit classes. Recognizing handwritten digits requires the network to understand patterns and edges which is relatively much more difficult for a simple single-hidden-layer feed-forward network.
To help the model perform better, particularly on the harder datasets (MNIST and Housing), I would consider the following architectural and training adjustments:

1. **Increase Training Epochs:** 10 epochs is an extremely short training duration. Allowing the model to train for a larger number of epochs would give slower learning rates the time they need to converge to a deeper minimum.

2. **Deepen the Network:** Adding additional hidden layers would allow the model to learn hierarchical feature representations. For MNIST, this could mean learning simple edges in the first layer and complex shapes in the subsequent layers.

3. **Optimizers:** Using a better optimizer to use an adaptive learning rate method like Adam would help the network navigate complex loss landscapes much more efficiently.

Part 5: Errors encountered

Bug 1: Swapped Arguments in Cross-Entropy Loss Evaluation

Output/Bug Printout:

```
--- Running Experiments for Iris ---
LR 1: Test Loss = 3.0765
Test Accuracy = 1.0000
LR 0.01: Test Loss = 23.0730
Test Accuracy = 0.1667
```

The test loss was reporting high numbers (e.g., 3.07) despite the model achieving perfect or reasonable accuracy. This occurred because I accidentally swapped the arguments when calling my loss function during the testing phase. My method signature was `compute_loss(self, y_true, y_pred):`, but in `hw1.py`, I was calling `test_loss = nn.compute_loss(y_pred, y_test)`.
I noticed that my training loss curves looked completely normal, while my printed test losses were suspicious. With this, I was able to figure out the issue correctly.

Bug 2: Standardization on the Iris Dataset

Output/Error Printout:

```
--- Running Experiments for Iris (Before Fix) ---
LR 1: Test Loss = 10.1542
Test Accuracy = 0.3667

--- Running Experiments for Iris (After Fix) ---
LR 1: Test Loss = 0.1588
Test Accuracy = 0.9333
```

Initially, the model trained on the Iris dataset with a learning rate of 1 was performing very poorly, getting stuck at around 36.67% accuracy (which is roughly equivalent to randomly guessing one of the three classes). The cause was that the raw input features (sepal and petal dimensions) were on different numerical scales. This caused the plain SGD optimizer to oscillate wildly and fail to converge to a good minimum.
I standardized the data and the loss became much more symmetric, allowing the learning rate of 1 to smoothly and rapidly converge to 96.67% accuracy.

Bug 3: NaN Gradients in the Housing Dataset

Output/Error Printout:

```
LR 1: Test Loss = nan, Train Loss = nan
RuntimeWarning: overflow encountered in square
RuntimeWarning: invalid value encountered in multiply
```

While training the neural network on the California Housing dataset with a learning rate of 1, the training and test losses suddenly started evaluating to NaN, even though I had successfully standardized the input features and the target values.
I tried the gradient clipping technique to restrict gradients between [-1,1] and this resolved the NaN errors.